

Adaptive AI Meal Planning for Obesity Prevention

Technical Specification: Architecture, Tech Stack, Roles & Implementation Roadmap

Project Focus: Adaptive, Evidence-Based Personalized Diet Engine	Team Structure: 4-Member Student/Research Group
Document Version: v1.0 (Production Architecture Spec)	Core Stack: Next.js, FastAPI, PostgreSQL, Scikit-learn, Gemini API

1. Executive Overview & Architectural Principles

Obesity prevention and weight management require long-term dietary compliance. Generic diet applications fail because they ignore individual preferences, meal fatigue, and behavioral patterns. Conversely, unconstrained LLMs hallucinate nutritional content, generate mathematically invalid macro totals, and present severe safety risks. This project builds an evidence-based, adaptive, and explainable personalized meal planning system where nutrition facts come strictly from validated datasets (USDA, ICMR-NIN) and AI is confined to personalization, re-ranking, and natural-language interaction.



PRIMARY ARCHITECTURAL PIPELINE

AUTHORITATIVE NUTRITION DATA (USDA/ICMR-NIN) → ETL & NORMALIZATION → STRUCTURED POSTGRESQL DB → HARD SAFETY GATE → HYBRID REC SCORING (0.30 Macro + 0.20 Goal + 0.20 Pref + 0.15 Feedback + 0.10 Diversity + 0.05 Prep) → EXPLAINABILITY (SHAP/Rules) → GEMINI LLM INTERFACE → USER FEEDBACK LOOP → ADAPTIVE RE-RANKING

- Rule 1: The LLM NEVER generates or alters calorie, macro, or medical facts.
- Rule 2: Hard safety constraints (allergies, restrictions) execute BEFORE recommendation scoring.

2. Final Recommended Tech Stack

The stack is selected to be modern, mostly open-source/free-tier, type-safe, and tailored to a 4-member research team.

Category	Technology	System Role	License / Cost	Team Skill Target
Frontend	Next.js (v14+) TypeScript Tailwind CSS	Server components, responsive UI, dashboard rendering, type-safe state.	Open Source / Free (MIT)	App Router, TS Interfaces, Tailwind utility layouts.
Backend	Python 3.11+ FastAPI	Async REST APIs, Pydantic validation, business logic, ORM routing.	Open Source / Free (MIT)	Async Python, Pydantic schemas, Dependency Injection.
Database	PostgreSQL (Supabase)	Relational recipe/food store, ACID compliance, JSONB logs, Row-Level Security.	Open Source / Free Tier	SQL schema design, foreign keys, RLS policies, Supabase SDK.
Auth	Supabase Auth	JWT authentication, user identity management, session hooks.	Managed Free Tier	JWT header validation in FastAPI, auth state hooks in React.

Data ETL	Pandas & NumPy	Ingestion, cleaning, deduplication, unit standardization (grams/kcal).	Open Source / Free (BSD)	DataFrame vectorization, missing value handling ('NaN').
Machine Learning	Scikit-learn	User preference regression, candidate meal re-ranking, clustering.	Open Source / Free (BSD)	Ridge/Logistic Regression, model pipelines, Precision@K.
Explainability	SHAP + Rules	Feature attribution calculation, deterministic rule justification generation.	Open Source / Free (MIT)	Tree/LinearExplainer, converting contribution scores to prose.
Generative AI	Gemini API	Natural language explanations, recipe instruction formatting, chat assistant.	Free Tier (Rate Limited)	System prompts, structured JSON parsing, context wrapping.
Analytics	Recharts	Interactive weight trend line charts, macro distribution pie charts.	Open Source / Free (MIT)	Dynamic SVG rendering, binding React state to charts.
Hosting	Vercel (FE) Render/Railway (BE)	Continuous deployment for Next.js frontend and Python backend API.	Free Tiers (Vendor subject)	Env variable management, build logs, deployment configs.

Key Architectural Justifications

Why Next.js? Client-side SPAs lack server rendering and type safety. Next.js App Router provides Server-Side Rendering (SSR) for fast dashboard loads, secure API key handling, and seamless TypeScript integration.

Why PostgreSQL over NoSQL? PostgreSQL enforces referential integrity between raw foods and composite recipes. NoSQL document stores duplicate ingredient data across recipes, leading to severe inconsistency when nutrition values update. PostgreSQL JSONB provides flexibility for AI logs while keeping relational safety.

Why FastAPI? FastAPI provides automatic Pydantic schema validation and OpenAPI docs, executing asynchronous recommendation logic far faster than synchronous frameworks like Django or Flask.

3. Four-Member Team Roles & Domain Ownership

To eliminate overlap and ensure clear accountability, responsibilities are divided into 4 isolated engineering domains.

Member Role	Core Focus	Target Tech	Primary Deliverables	GitHub Domain
Member 1: Frontend Lead	UI/UX, Client State, Dashboard Analytics	Next.js, TypeScript, Tailwind CSS, Recharts	Multi-screen UI, onboarding wizard, meal plan dashboard, feedback forms.	/frontend
Member 2: Backend Lead	REST API, Database Architecture, Security	FastAPI, PostgreSQL, Supabase Auth, Pydantic	Async API routes, SQL schemas, JWT security middleware, database RLS.	/backend
Member 3:	Rec Engine, Adaptive	Scikit-learn, SHAP,	Candidate scoring engine,	/ml

AI/ML Lead	Models, XAI, LLM	Gemini API, NumPy	preference re-ranking loop, XAI pipeline, prompts.	
Member 4: Data Lead	Data Pipeline, Nutrition Sourcing, Safety	Pandas, NumPy, USDA/ICMR Datasets, SQL	Cleaned seed dataset, ETL pipeline scripts, hard constraint safety gate.	/data /scripts /tests

4. Phased Implementation Roadmap (20-Week Plan)

A structured 10-phase execution schedule designed to deliver a fully functional research prototype.

Phase	Milestone Name	Timeline	Core Deliverables & Tasks	Lead Owner
Phase 1	Requirements & Specs	Weeks 1–2	Define system architecture, database schemas, and Git workflows.	All Members
Phase 2	Data Extraction & ETL	Weeks 3–4	Ingest USDA & ICMR datasets; clean, normalize units (kcal/grams), deduplicate.	Member 4
Phase 3	Database & Auth Setup	Weeks 5–6	Deploy Supabase PostgreSQL instance; execute SQL schemas & configure RLS.	Member 2
Phase 4	Backend API Foundation	Weeks 7–8	Build FastAPI core endpoints (`/auth`, `/foods`, `/recipes`, `/progress`).	Member 2
Phase 5	Frontend UI Foundation	Weeks 9–10	Construct Next.js pages, Tailwind components, and API integration hooks.	Member 1
Phase 6	Rule-Based Rec Engine	Weeks 11–12	Implement hard constraint safety gate and multi-factor mathematical scoring logic.	Member 3
Phase 7	Adaptive Feedback Loop	Weeks 13–14	Build post-meal evaluation routes and dynamic preference weight update engine.	Member 3 & 2
Phase 8	XAI & Gemini Integration	Weeks 15–16	Integrate SHAP feature attributions, configure Gemini API, format explanations.	Member 3
Phase 9	End-to-End System Sync	Weeks 17–18	Integrate Next.js frontend with FastAPI backend; conduct integration testing.	All Members
Phase 10	Deployment & Evaluation	Weeks 19–20	Deploy to Vercel/Render, execute user testing, measure Precision@K and NDCG.	All Members

5. Free / Low-Cost Infrastructure Allocation

Infrastructure Tool	Selected Platform	Free Tier Allotment / Limitation	Risk Mitigation Strategy
Frontend Hosting	Vercel	Hobby Tier: 100 GB Bandwidth, automated GitHub deployments.	Migrate to Netlify / Cloudflare Pages if bandwidth limits are

			reached.
Backend Compute	Render / Railway	Free / Low-cost trial compute instances.	Standard Dockerized ASGI app allows migration to any cloud host.
Managed Database	Supabase (PostgreSQL)	Free Tier: 500 MB DB storage, 50k monthly active users.	Export regular SQL backups to allow instant local migration if needed.
Generative AI API	Gemini API	Free Tier available (Subject to rate limits).	Implement local string template fallbacks if API rate limit is exceeded.
Code Management	GitHub	Free tier for private repositories and GitHub Actions CI/CD.	Standard Git architecture ensures local code safety across all team members.